

Fresh JMLR MLOSS reviewer report: kodama-cpp

Independent first-read assessment of the current submission package

Review date: 19 July 2026

Summary

This submission presents a substantial standalone implementation of KODAMA rather than a new learning objective. The manuscript clearly distinguishes continuity with the published KODAMA method from the new C++17 systems contribution: float32 state ownership, CPU/CUDA/Metal backends, label-aware SIMPLS plus latent-space LDA, package-owned neighbor search, supplied-graph entry points, and thin R/Python bindings. The algorithm is reproducible from the equations and pseudocode: proposal generation, one-CV-pass-per-cycle evaluation, acceptance score, cooling rule, independent M runs, landmark and splitting semantics, projection, and agreement-graph correction are all specified.

The empirical section is appropriately cautious. Internal CV accuracy is not presented as external truth, reference labels are used only after optimization, and positive and negative visualization results are retained. Kernel, optimizer, and end-to-end timings are separated. A legacy classifier-matched comparison and a separately scoped current-CRAN systems comparison prevent the predecessor discussion from relying on an obsolete release alone.

Major Comments

I identify no remaining major scientific or software-methods objection within the declared scope. The architecture is non-trivial, the public objective is fully described, the software is testable, and the manuscript does not claim universal improvement over standard visualization or clustering.

Minor Comments

1. Create the clean v0.1.0 tag, archive that exact commit, and insert the archive digest and DOI before submission. The manuscript correctly leaves these fields unresolved instead of inventing them, but a software paper should cite an immutable artifact.
2. Complete the author-only cover-letter fields: coauthor consent, funding, competing interests, conflict-free editor/reviewer suggestions, and dated community metrics. These are submission formalities rather than defects in the software contribution.
3. Keep the current-CRAN comparison explicitly labeled as a short end-to-end systems check. KODAMA 3.3 automatically chooses its PLS route and is not classifier- or trajectory-parity with kodama-cpp PLS-LDA. The full $M=100/T_{\text{cycle}}=100$ legacy and sensitivity evidence should remain separate.
4. Document the native CUDA graph-builder limit $k \leq 256$ prominently. Current CRAN couples a large landmark request to $k=500$ on MetRef, so an exactly matched CUDA row is correctly omitted rather than produced with a silent parameter change.
5. The measured CPU coverage is useful but moderate (63.58% lines and 57.03% branches). Graph clustering and several PLS helper paths are the clearest future test targets. CUDA and Metal hardware tests should continue to be reported separately rather than folded into the CPU percentage.
6. Most reported wall times are descriptive runs, not replicated performance distributions. Preserve that language and avoid inferential speed claims until repeated measurements across additional hardware are archived.

Strengths

- The distinction between raw CV accuracy, proposal acceptance score, and external diagnostics is unusually clear and prevents circular quality claims.
- The single-run and ensemble pseudocode make the stochastic optimization independently implementable.
- Backend identity is strict and testable; unavailable accelerators do not masquerade as successful CPU execution.
- The evidence matrix links implementation claims to source locations, tests, and measured results.

- Cross-platform GitHub Actions, measured CPU coverage, a frozen API test, provenance audit, and hosted documentation substantially strengthen MLOSS readiness.
- The manuscript retains unfavorable datasets and states that KODAMA correction is classifier- and dataset-dependent.

Reviewer Recommendation

Recommendation: Minor Revision. The manuscript and software are suitable in scope and technical substance for JMLR MLOSS. The remaining requests concern release immutability, author declarations, and sharper disclosure of benchmark and CUDA graph-size boundaries; they do not require a new algorithm, a redesign of the implementation, or a major new experimental campaign.

I would not recommend rejection or major revision on the basis of the limited current result set: the paper already contains named multi-dataset positive and negative evidence, parameter sensitivity, cross-platform validation, and a carefully bounded claim. Broader repeated benchmarks would increase impact, but the present MLOSS contribution is primarily the reusable heterogeneous implementation.